

113-2 生物光機電系統技術與應用

Noise2Void 於雞舍影像品質改善

組別：10

	學號	姓名
組員1	B11611051	鄧杰修
組員2	B11611012	陳子員
組員3	B11611027	李傳漢

一、摘要

在本期末專題中，我們探討了如何利用深度學習模型進行影像去雜訊處理，尤其聚焦於僅需單一雜訊影像即可訓練的 Noise2Void (N2V) 方法。本報告整理並實作了 N2V 架構，評估其在雞舍監控影像去雜訊上的表現。實驗結果顯示，N2V 能在無清楚影像標註的資料下進行有效學習。本研究同時比較了完整影像與 patch 分割訓練方式的異同，並討論此模型於實際部署時可能面臨的效能瓶頸與潛在應用延伸。

Keywords: image denoising, self-supervised learning, Noise2Void, chicken house monitoring, biomedical imaging

二、引言

影像去雜訊技術在電腦視覺與生物醫學影像處理中扮演關鍵角色，其目的在於去除感測器或環境造成的雜訊，進而提升圖像品質與後續分析任務的準確性。近年來，隨著深度學習的發展，多種學習式去雜訊模型相繼問世，如需配對資料的 DnCNN、Noise2Noise 等方法。然而乾淨影像標的往往難以取得，甚至無法重複拍攝同一目標，限制了傳統方法的應用範圍。

本研究選擇了 Krull 等人於 2019 年提出的 Noise2Void 模型[1]，該模型透過 blind-spot 網路與自我監督式訓練策略，僅需單一雜訊影像即可進行有效訓練，極大拓展了其在實際應用中的彈性與可行性。

三、文獻回顧

i) 研究目的與問題

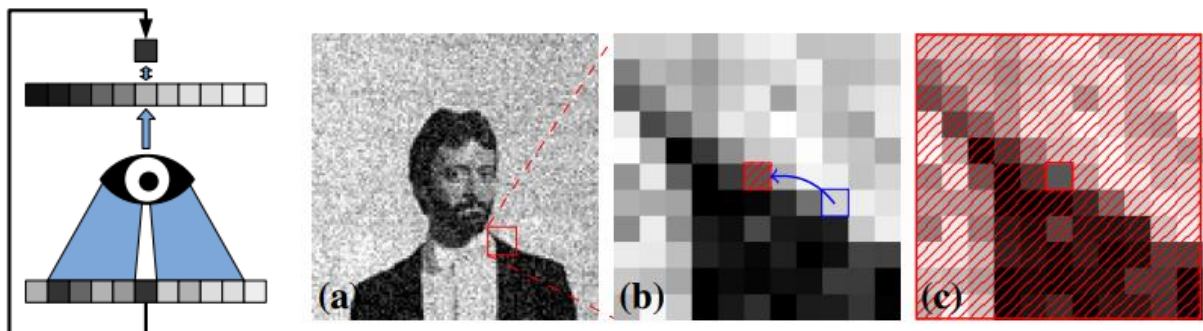
本研究旨在解決影像去雜訊 (denoising) 時所面臨的一項關鍵挑戰：缺乏乾淨的訓練標的影像。傳統的深度學習去雜訊方法需依賴成對的「雜訊影像」與「其對應乾淨影像」，或至少需有兩張具有相同內容但雜訊不同的影像對，例如 Noise2Noise (N2N) 方法。

然而，在許多應用場域 (尤其是生醫影像) 中，這類資料難以取得甚至無法獲得。因

此，本研究提出一個不需要任何乾淨或配對影像的自我監督學習方法——Noise2Void (N2V)，以解決在資料有限的情況下進行影像去雜訊的問題。

ii) 研究方法與設計

研究採用了盲點網路 (blind-spot network) 的設計，使得每次預測一個像素時，該像素本身的值會從輸入中被遮蔽，僅使用其周圍像素來推測中心值。訓練過程中採用隨機遮蔽 (random masking) 策略，在輸入影像中隨機挑選像素遮蔽其值，並以遮蔽前的原始值作為目標值。這樣的設計避免網路學習影像自身 (identity mapping)，並利用影像內部統計關係進行學習。整體方法僅需一張含雜訊的影像進行訓練，屬於自我監督 (self-supervised) 訓練方式。



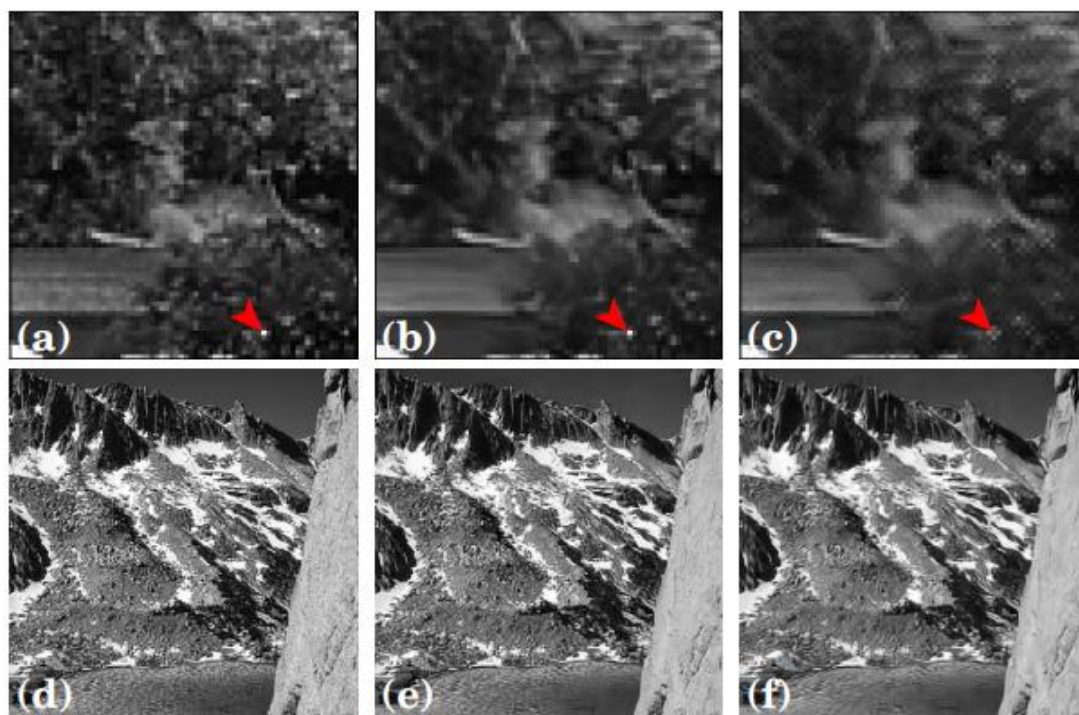
圖一、blind-spot 遮蔽策略示意圖：(a) 原始雜訊影像；(b) 放大區域中隨機選取一個像素（紅框），並以鄰近像素值（藍箭頭）覆蓋該像素值，製造 blind-spot；(c) 遮蔽後的輸入影像用於訓練，僅在遮蔽像素位置計算 loss。

iii) 主要發現與結果

1. 在自然影像（如 BSD68）上，N2V 雖無法超越傳統 supervised 或 N2N 訓練方式的表現，但仍優於非學習式方法如 BM3D、mean/median filter。
2. 在模擬生醫影像上，N2V 的效能與傳統方法及 N2N 幾乎持平。
3. 在真實生醫資料（如 cryo-TEM 與 fluorescence microscopy）中，N2V 是唯一可應用的方法，並且能在無任何標的影像的情況下達到實用的去雜訊效果。
4. 速度上，訓練後的 N2V 推論顯著快於 BM3D 等傳統方法（約25倍）。
5. 方法限制在於：若影像噪聲具結構性或影像中含不可預測的獨立亮點，效果會明顯下降，如圖三所示：

	Ground Truth	Input	BM3D	Traditional	NOISE2NOISE	NOISE2VOID
BSD68			 PSNR: 28.59	 PSNR: 29.06	 PSNR: 28.86	 PSNR: 27.71
Simulated Data			 PSNR: 29.96	 PSNR: 32.56	 PSNR: 32.43	 PSNR: 32.28
cryo-TEM	 Does not exist.		 Runtime: ~33.2s	 Clean target not available.	 Runtime: ~1.3s	 Runtime: ~1.3s
CTC-MSC	 Does not exist.		 Runtime: ~4.6s	 Clean target not available.	 Noisy target not available.	 Runtime: ~0.1s
CTC-N2DH	 Does not exist.		 Runtime: ~5.2s	 Clean target not available.	 Noisy target not available.	 Runtime: ~0.1s

圖二、不同影像去雜訊方法在多種資料集上的視覺與數值比較



圖三、圖 5. NOISE2VOID 在困難影像上的預測失敗案例

(a - c) 顯示一張包含單獨高亮像素的影像去雜訊結果。紅色箭頭指出 N2V 未能成功預測的區域：(a) 原始影像片段；(b) 傳統訓練的結果；(c) N2V 訓練的結果，亮點資訊遺失。(d - f) 顯示一張高頻細節豐富的影像去雜訊結果：(d) 原圖；(e) 傳統訓練結果；(f) N2V 結果，明顯損失更多細節。這些案例突顯 N2V 在處理不可預測訊號（如孤立亮點或紋理豐富區域）時的限制。

iv) 論文結論與建議

作者成功提出一種僅需單一雜訊影像就能訓練的去雜訊方法，克服了傳統方法在資料蒐集上的限制，尤其適用於無法取得標的影像的領域（如生醫影像）。N2V 建立在合理的統計假設下（信號具統計依賴性，雜訊具條件獨立性），並透過創新的 blind-spot 網路設計實現有效去雜訊。雖然在某些極端情況下表現不佳，但整體而言提供了一個實用且通用的去雜訊解法。未來建議可針對結構性雜訊進行改進，或結合其他補充訊息（如時間序列、空間一致性）來強化模型效果。

四、 材料及方法

A. 實驗材料：

- 個人電腦 * 1

B. 實驗對象：

- 畜牧場雞隻(台灣土雞，約12週齡)

C. 實驗方法：

使用 <https://github.com/juglab/n2v> 上的模型進行訓練。訓練分成單一影像作為資料以及切割成多個 Patches 作為資料

D. 實驗步驟：

1. 先把套件以及模型匯入 vscode 之中

```
# 🚀 Step 1: Import libraries
import os
import numpy as np
from skimage.io import imread, imsave
from matplotlib import pyplot as plt

from csbdeep.utils import normalize
from n2v.models import N2V, N2VConfig
import tensorflow as tf
print("Eager mode enabled?", tf.executing_eagerly())

gpus = tf.config.list_physical_devices('GPU')
print("Available GPUs:", gpus)
```

```
Eager mode enabled? True
Available GPUs: []
```

2. 將影像存入資料夾中並載入與轉換影像資料

```
import os
import numpy as np
from skimage.io import imread
from skimage.transform import resize

image_dir = r'Z:\Workspace\Jay\n2v_project\images_one'
image_list = sorted([f for f in os.listdir(image_dir) if f.endswith(('.png', '.jpg', '.tif'))])
imgs = []

target_shape = (1088, 1920) # 預設輸出大小，需能被 4 整除

for fname in image_list:
    img_path = os.path.join(image_dir, fname)
    img = imread(img_path)

    # 處理灰階圖
    if img.ndim == 2:
        img = img[..., np.newaxis]

    # 處理格式錯誤的彩色圖，例如 (C, H, W) 或 (H, C, W)
    elif img.ndim == 3:
        if img.shape[0] in [1, 3] and img.shape[0] != max(img.shape): # (C, H, W)
            print(f"🔄 Transposing (C, H, W) → (H, W, C): {fname}")
            img = np.transpose(img, (1, 2, 0))
        elif img.shape[1] in [1, 3] and img.shape[1] != max(img.shape): # (H, C, W)
            print(f"🔄 Transposing (H, C, W) → (H, W, C): {fname}")
            img = np.transpose(img, (0, 2, 1))

    # Resize 成固定大小 (H, W) 並保留 channel 數
    resized = resize(img, (*target_shape, img.shape[-1]), preserve_range=True, anti_aliasing=True)
    resized = resized.astype(np.float32) # 統一格式，防止預測錯誤

    imgs.append(resized)

X = np.array(imgs)
Y = X.copy()

print(f"✅ Loaded {X.shape[0]} images. Final shape: {X.shape}")
```

✅ Loaded 1 images. Final shape: (1, 1088, 1920, 3)

3. 正規化影像資料

```
# ✅ Normalize images
X = normalize(X, 1, 99.8, axis=(1, 2, 3))
```


4. 設定 N2V 模型參數

```
# ✂ Step 3: Configure model
config = N2VConfig(
    X,
    unet_kern_size=3,
    train_steps_per_epoch=100,
    train_epochs=10,
    train_loss='mse',
    batch_norm=True,
    train_batch_size=4,
    n2v_perc_pix=0.198,
    n2v_patch_shape=(64, 64),
    n2v_neighborhood_radius=5
)
```

5. 建立模型資料夾並訓練模型

```
# ...existing code...
import os
import shutil
import time
from n2v.models import N2V

# ✔ 用全新目錄名稱，避開已污染舊資料夾
model_name = 'N2VModel_v4_one'
model_path = os.path.join('models', model_name)

# 若存在同名資料夾或檔案，直接移除
if os.path.isfile(model_path):
    os.remove(model_path)
elif os.path.isdir(model_path):
    shutil.rmtree(model_path)
time.sleep(1)

# 確保 models 資料夾存在
os.makedirs('models', exist_ok=True)
# 建立乾淨資料夾
os.makedirs(model_path, exist_ok=True)
print(f"✔ 已建立 {model_path}")

# 建立模型並訓練
model = N2V(config=config, name=model_name, basedir='models/')
model.train(X, Y)
# ...existing code...
```

✔ 已建立 models\N2VModel_v4_one
z:\Workspace\Jay\n2v_project\n2v\models\n2v_standard.py:447: UserWarning: output path for model already exists
warnings.warn(
8 blind-spots will be generated per training patch of size (64, 64).
Preparing validation data: 100%|██████████| 1/1 [00:00<00:00, 6.42it/s]
Epoch 1/10
1/1 [=====] - 3s 3s/steps - loss: 0.5024 - n2v_mse: 0.5024 - n2v_abs: 0.49
100/100 [=====] - 27s 221ms/step - loss: 0.5024 - n2v_mse: 0.5024 - n2v_abs: 0.4985
Epoch 2/10
1/1 [=====] - 2s 2s/steps - loss: 0.2758 - n2v_mse: 0.2758 - n2v_abs: 0.37
100/100 [=====] - 20s 197ms/step - loss: 0.2758 - n2v_mse: 0.2758 - n2v_abs: 0.3761
Epoch 3/10

6. 儲存模型

```
...
1/1 [=====] - 3s 3s/steps - loss: 0.1879 - n2v_mse: 0.1879
100/100 [=====] - 22s 216ms/step - loss: 0.1879 - n2v_mse:

Loading network weights from 'weights_best.h5'.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

<keras.src.callbacks.History at 0x1b0e53ca5b0>
```

7. 對第一張影像進行預測

```
# 🧠 Step 5: Predict on the first image
denoised = model.predict(X[0][np.newaxis, ...], axes='SYXC')

1/1 [=====] - 3s 3s/step
```

8. 顯示原圖與去噪結果

```
# 🖼️ Step 6: Show comparison
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.title("Original")
plt.imshow(X[0].squeeze())
plt.axis('off')

plt.subplot(1,2,2)
plt.title("Denoised")
plt.imshow(denoised[0].squeeze())
plt.axis('off')
plt.show()
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers)

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers)



9. 把去噪結果存入輸出資料夾

```
# 📁 Step 7: Predict and save all RGB images
import os
from skimage.io import imsave

os.makedirs('output_one_3', exist_ok=True)
for idx, img in enumerate(X):
    # 預測
    denoised = model.predict(img[np.newaxis, ...], axes='SYXC') # shape: (1, H, W, C)

    # 不使用 squeeze，以免壓掉 RGB 的 channel
    denoised_img = denoised[0] # shape: (H, W, 3)

    # 正規化回 0~255 並轉為 uint8
    img_to_save = (denoised_img * 255).astype(np.uint8)

    # 檔名與輸出
    orig_filename = image_list[idx]
    output_path = os.path.join('output_one_3', f'denoised_{orig_filename}')
    imsave(output_path, img_to_save)

    print(f"📁 Saved to {output_path}")
```

1/1 [=====] - 3s 3s/step

📁 Saved to output_one_3\denoised_20250501_1836_0000.png

10. 重複步驟並找出最佳的 Hyper Parameter

E. 結果

	1	2	3
原圖			
降噪後			

表一、小範圍 patch 降噪結果與原圖比較

原圖	降噪後
	

表二、大範圍降噪結果與原圖比較

五、 結果與討論

1. 在實驗中我們觀察到，經過 N2V (Noise2Void) 模型處理後的影像具有顯著的去噪效果。這項技術透過預測每個像素周圍的鄰近像素來進行降噪，因此能有效地去除隨機性雜訊同時保留原始結構細節。在我們的應用場景中，原始影像色調整體偏藍，而在進行 N2V 處理前，我們還對影像進行了強度正規化的操作（採用 1% 至 99.8% 的 percentile 範圍），可能進一步強化了藍色通道的視覺強度。儘管部分去噪後的影像在視覺上呈現出比原圖更偏藍的色彩，但這並不影響我們的目標——提升物體偵測的準確性與穩定性。實驗結果顯示，經過 N2V 處理後的影像在視覺上更加清晰，背景干擾減少，邊界輪廓變得更加明顯，這使得不論是人眼檢視還是後續以 YOLO 模型進行自動物件偵測時，都能更精確地辨識出雞隻的位置與形態。
2. 分割成許多 patches 的結果其實會比單一照片的效果更好，因為 N2V 預測目標僅依賴鄰近像素資訊（即局部性強），適合在小 patch 上學習，並且當切成小 patches 的時候會有較佳的泛化能力。
3. 整個結果的出現需要三分鐘，若給予太多資料的時候會給予太多資料的時候會容易導致電腦當機以及記憶體被占滿。

六、 結論與建議

1. N2V 模型具備明顯的去噪效果：透過預測每個像素周圍鄰近像素值，成功保留原始結構並去除隨機雜訊。即使影像原始色調偏藍，經正規化後的去噪結果仍有效提升視覺品質。
2. 切割成 Patches 訓練效果優於整張影像：相較於單張完整影像進行訓練，將影像切割成小 patch 再進行訓練能大幅提升泛化能力與準確度，這與 N2V 模型本身依賴局部資訊的特性相符。
3. 耗時：當前所使用的 N2V 模型在進行完整的影像去噪處理時，單張影像平均約需三分鐘的運算時間。這樣的處理時間在高解析度影像或批次資料量龐大的情況下，會提升整體處理成本與系統延遲，對於即時應用或大規模部署而言，會造成明顯的效能瓶頸。儘管模型本身在去噪品質上表現出色，但其高計算成本仍是一項挑戰。
4. 未來發展：
 - i. 結合即時物件偵測系統 (YOLO) 進行前處理整合，將 N2V 模型作為 YOLO 等物件偵測系統的前處理模組，使降噪與偵測能夠無縫串接，打造完整自動標記系統。
 - ii. 除雞舍監控外，可應用於水稻病斑、果實成熟度、葉片損傷等農業影像場景，可使用 N2V 去噪對不同影像進行訓練前處理。

七、 參考文獻

[1] Krull, A., Buchholz, T.-O., & Jug, F. (2019). Noise2Void - Learning denoising from single noisy images. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2129–2137. <https://doi.org/10.48550/arXiv.1811.10980>

八、 組員分工模式

	影像蒐集	模型訓練	論文閱讀與整理	報告架構與排版	簡報製作
鄧杰修		√		√	
陳子員	√		√		
李傳漢			√		√